Real-world (a tool backed by a real API call) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper LangChain/Python). Read Basic first even if you already know this — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — The idea, and the exact pieces

Basic Version

Your first tool (from the Basic exercise) just did math — no outside call, nothing could really go wrong. A real tool talks to a real service, and real services time out, go down, or send back something unexpected.

You already built a small HTTP client with retry logic back in Doc02. Reuse it here, instead of calling `requests.get(...)` directly inside your tool — the whole point of this exercise is putting that old retry logic somewhere new: behind a tool the model can call.

Pick any free public API that needs no signup (a weather API or a currency-conversion API both work fine).

Things to use:

- `from http_client import request_with_retry` (or however you named Doc02's client — import it, don't rewrite it).
- `@tool` from `langchain_core.tools`.
- A free API with no signup — for example `https://api.open-meteo.com/v1/forecast` (weather, no key needed).
- Build the URL/params yourself inside the tool function, using the argument the model sent.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Doc02's `request_with_retry()` already handles the messy parts (timeouts, transient failures) — don't rewrite that logic inside the tool function. Instead, call it *from* the tool function, and let the `@tool` decorator wrap the whole thing.

Keep the argument shape small and typed (a single typed parameter like `city: str` is fine to start) — the model needs to know exactly what to send, and Doc02's client needs a real URL built from that argument.

The tool function should stay thin: build the request URL/params, call `request_with_retry("GET", url, params=params)` — Doc02's signature takes the HTTP method first — and return the one or two fields the model actually needs, not the whole raw JSON blob. In Basic, let a real failure raise, so you see what an unhandled tool error looks like once. In Intermediate, catch Doc02's `TransientHTTPError` and `PermanentHTTPError` by name and return `f"Error: ..."` instead.

Write out, in plain words, the full call chain: model decides to call the tool → your tool function runs → it calls `request_with_retry()` → that makes the actual HTTP call → the result flows back up through all three layers.

Difference between Basic and Intermediate: same underlying tool (build params, call the API, return a summary) at 2 completeness levels — Basic gets one working call end to end, Intermediate splits the params-building out into its own testable function and checks `response.tool_calls` before looping. Both trust the response shape completely and only support the 2 hardcoded cities — worth knowing that's a demo shortcut, not something this exercise needs you to fix.

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
define a tool called get_weather that takes a city name

inside it:
    build the URL/params for the weather API
    call request_with_retry with that URL
    pull out the temperature from the response
    return a short text summary

register the tool on a model

ask "what's the weather in Lahore?"

print the tool call, then actually run it and print the real result
```

Here is almost the whole thing, missing the actual API call:

```
# real_api_tool_practice.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from http_client import request_with_retry  # your Doc02 client

@tool
def get_weather(city: str) -> str:
    """Get the current temperature for a named city."""
    # TODO: build the params, call request_with_retry, pull out the temperature
    pass

model_with_tools = ChatOpenAI().bind_tools([get_weather])
```

Fill in the `TODO`, then run it and check against the Solution.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

One piece to get you unstuck — the shape of the params dict and the return value:

```
# real_api_tool_practice.py
WEATHER_URL = "https://api.open-meteo.com/v1/forecast"

CITY_COORDINATES = {
    "lahore": {"latitude": 31.55, "longitude": 74.34},
    "karachi": {"latitude": 24.86, "longitude": 67.01},
}

def get_weather(city: str) -> str:
    coords = CITY_COORDINATES.get(city.lower())
    if coords is None:
        return f"No coordinates on file for {city}."
    params = {
        "latitude": coords["latitude"],
        "longitude": coords["longitude"],
        "current_weather": True,
    }
    # TODO: call request_with_retry("GET", WEATHER_URL, params=params),
    # pull out
    # ["current_weather"]["temperature"], and return a text summary.
```

Finish the `TODO` yourself, wire it up to a model with `@tool` and `bind_tools`, then compare against the Solution.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution